StreamCart

A real-time e-commerce intelligence platform

What the project solves, the value it delivers, how the architecture works, and what every tool in it is for and why it was chosen.

At a glance	
The business	A simulated global online shop selling in five markets: the US, Europe, Asia, Africa and Latin America
The data	Orders, website clicks and warehouse stock movements, about 50,000 to 75,000 orders a day, plus real exchange rates and real weather
What it delivers	Fraud and low-stock alerts within seconds, and validated, trustworthy business reporting from one shared source of data
How it's built	Kafka, ksqlDB, Spark, Apache Iceberg on MinIO, Great Expectations, dbt, Airflow, Postgres, Grafana, Superset and FastAPI, in 24 Docker containers started with one command
Status	All requirements built and tested from a clean install, with 104 automated tests and CI on every push

Repository: github.com/Jerry-Khobby/StreamCart-Real-Time-E-Commerce-Intelligence-Platform

Contents

1. The business problem
2. The value it delivers
3. The data
4. The architecture in one picture
5. Following one order through the system
6. The tools: what each does and why it was chosen
7. The data model
8. Keeping the data trustworthy
9. Running reliably
10. Security
11. What it looks like
12. Results from testing
13. Limits, and what production would change
14. Glossary

1. The business problem

Picture a growing online shop that sells electronics, clothing, jewellery, sports gear and homeware to customers in five markets. Every second, things happen on its website and in its warehouses: someone searches for a product, someone adds a pair of trainers to their basket, someone pays $900 for a laptop, a warehouse in Lagos ships three units of a blender and drops below its restock level. Each of these is a small piece of data, and together they are millions of events a day.

The company has the data, but it can't use it well. Three problems keep costing it money:

Problem	What it looks like day to day	What it costs
Fraud is caught too late	Fraud checks run in a nightly batch report. A stolen card is used at 10 a.m., and the report flags it the next morning, after the goods have shipped.	Chargebacks, lost stock and payment-provider penalties. Every hour of delay is money already gone.
Stock decisions use yesterday's numbers	Warehouse teams plan restocking from a report built overnight. A product that sells out at noon isn't noticed until tomorrow.	Empty shelves for best sellers, lost sales, and over-ordering of slow lines.
Every team has its own numbers	Marketing, finance and operations each pull data their own way. Revenue for last Tuesday differs depending on who you ask.	Time wasted reconciling figures, and decisions made on numbers nobody fully trusts.

The two kinds of need pull in opposite directions. A fraud alert must be **fast**, even if the data isn't perfect yet. A finance report must be **correct**, even if it is an hour old. A single pipeline tuned for one does the other badly. StreamCart's central design idea is to take **one stream of events** and serve both needs through **two paths**, so that speed and accuracy each get a path built for them, and both paths start from the same data.

2. The value it delivers

Put simply: *we have data coming from an e-commerce site, and we turn it into decisions people can act on, at the speed each decision needs.*

Who	What they get	Where	How fresh
Fraud analysts	A live feed of suspicious orders, labelled with the reason: a very large order right after login, a burst of orders from one customer, or an order from another country	Grafana	Seconds
Warehouse teams	Stock levels as they change, and an alert the moment a product drops below its reorder point	Grafana and the REST API	Seconds
Finance	Daily revenue by market in US dollars, converted at the exchange rate in force when each order was placed, with refunds separated out	Superset	Hourly
Marketing	Where visitors drop out between viewing a product and paying, which customers come back, and the best-selling products	Superset	Hourly
Data and engineering teams	Proof that the data is healthy: job success rates, row counts, freshness, quality checks, schema changes and alerts when something breaks	Grafana and a chat webhook	Every 15 min
A fraud model (software)	A customer's recent order count, spend, markets and alerts, ready for a machine learning model to score	REST API	Seconds

> **The business value in one line.** Fraud is flagged while it can still be stopped, warehouses act on today's stock rather than yesterday's, and every team reads revenue from the same validated tables, so the numbers agree.

Why one platform rather than separate tools

The alerts and the reports start from the same events. Because there is a single source of truth, the fraud count in the live dashboard and the fraud summary in the monthly report are built from the same orders. When they differ, it is because the report has been cleaned and validated, and the platform can show exactly which rows were rejected and why.

3. The data

There is no real shop behind this project, so a **simulator** plays the part of the website, the warehouses and the customers. It is built to behave like a real business, including the messy parts, so that every later stage has real work to do. Exchange rates and weather come from real public APIs.

Source	What it produces	How it arrives
Website orders	Each order: customer, product, market, currency, amount, payment method, status, and how long the customer had been logged in	Streamed to Kafka as it happens
Clickstream	Page views, searches, product views, add-to-basket, checkout, and abandoned baskets, grouped into visitor sessions	Streamed to Kafka
Warehouse stock	Every stock movement: orders shipped, restocks, damaged goods written off, audit corrections	Streamed to Kafka
Transactional database	The shop's own records of users, orders and payments, as a storefront would keep them	Pulled daily by Airflow
Exchange rates	Real rates from open.er-api.com for USD, EUR, JPY, NGN and BRL	Fetched hourly
Weather	Real temperature and rainfall for a city in each market, from Open-Meteo	Fetched hourly
Inventory API	Current stock per product and warehouse, over REST	Pulled hourly

What makes the simulated data realistic

- **Volume and rhythm:** about 52,600 orders on a Sunday and 75,000 on a weekday, with morning and evening peaks, a night-time trough and weekend dips.
- **Seasons:** category demand changes by month, such as toys peaking in December.
- **Weather:** a market where it's raining gets 25% more orders, using the real weather feed.
- **Fraud:** card testing (a burst of small charges), account takeover (a very large order seconds after login) and geo anomalies (an order from outside the customer's country). About 30% of the fraud is designed to slip under the rules, and some innocent orders look suspicious, so the fraud rules are tested honestly.
- **Bots:** 5% of website sessions are bots. Half admit it in their user agent; half pretend to be a browser and can only be caught by how fast they click.
- **Bad data:** about 1% of orders are deliberately corrupted (missing customer, negative amount, invalid currency, impossible timestamp), and 1% are sent twice, as a network retry would.
- **History:** on first start, 14 days of past orders are generated so the reports have trends to show.

4. The architecture in one picture

The diagram on the next page is turned sideways so it can be printed large enough to read. Orange is the speed path (seconds), blue is the batch path (hourly and daily), and grey dashed lines are scheduled or supporting flows.

The two paths

The speed path (seconds). Events land in Kafka. ksqlDB watches the order and stock streams and applies the fraud and low-stock rules to every event as it arrives, publishing matches back to Kafka as alerts. A small Python service, the serving sink, copies orders, stock and alerts into Postgres, where Grafana's live dashboards and the REST API read them. An order typically reaches the dashboard about a second after it happened.

The batch path (hourly and daily). Spark copies every event from Kafka into the **Bronze** layer of a data lakehouse, exactly as it arrived. Every hour, a Spark job cleans and validates Bronze into **Silver**: it rejects bad rows, removes bots and duplicates, stitches visits into sessions and converts every amount to US dollars. When Silver is updated, dbt builds **Gold**: a star schema of business tables and KPIs, tested before anyone can see them. Superset reads Gold for business reporting.

The medallion layers

Layer	What's in it	Rule	Stored in
Bronze	Every event exactly as it arrived, plus when it arrived and its Kafka position	Never change it. It's the record to replay from if a rule was wrong	Apache Iceberg on MinIO
Silver	Clean, validated, de-duplicated events in US dollars, with bots removed and sessions stitched	Only rows that pass every quality check. Rejected rows are kept separately with the reason	Iceberg on MinIO, and a copy in Postgres for dbt
Gold	Business tables: customers, products, dates, markets, orders, sessions, stock, and ready-made KPIs	Only published after every test passes	Postgres

Orchestration and observability

Apache Airflow runs everything that happens on a schedule: the hourly Silver job, the daily database pull, exchange-rate and weather fetches, the Gold build, nightly maintenance and a monitor that checks the pipeline's health every 15 minutes. Every failure is recorded for Grafana and can be posted to a chat channel.

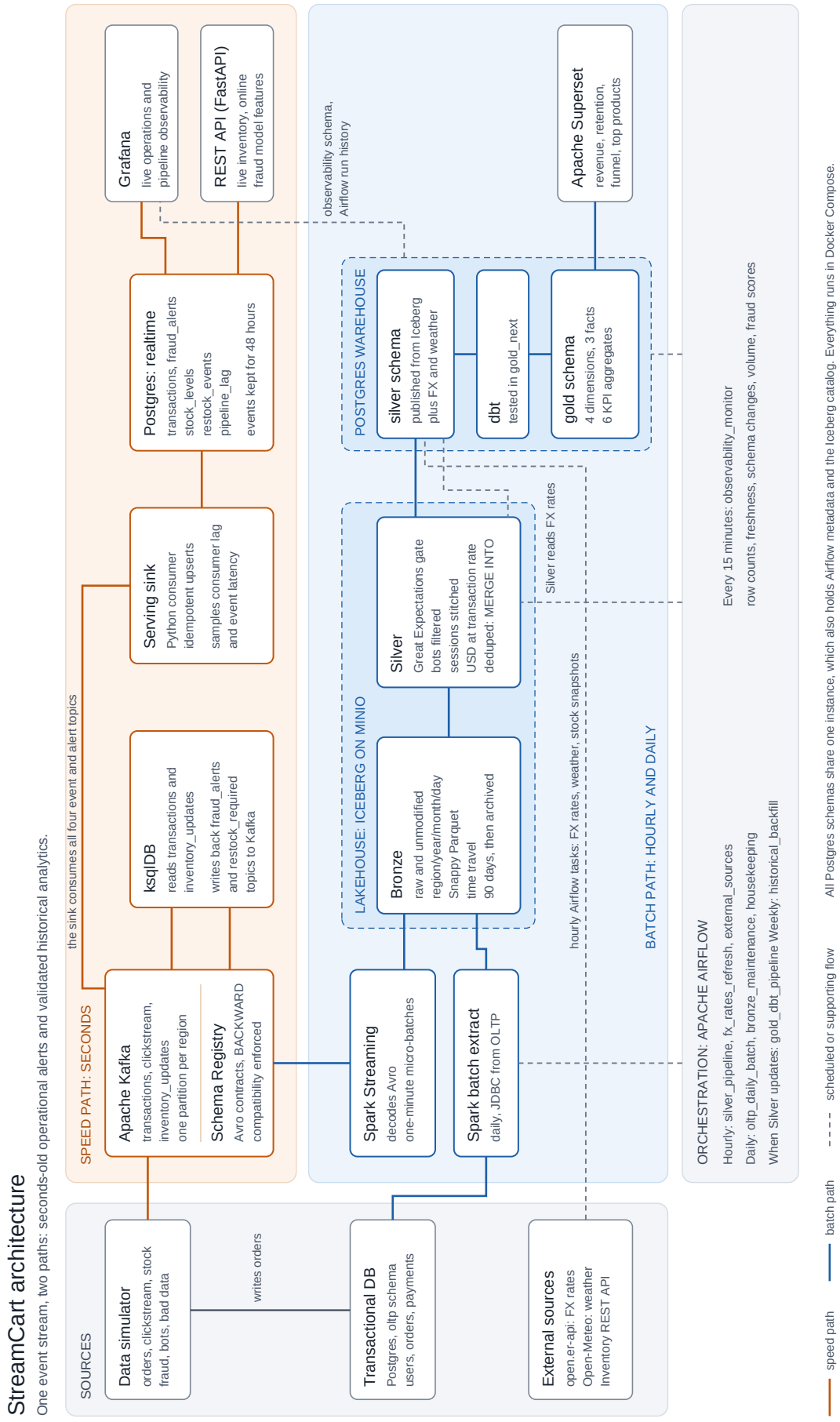


Figure 1. One event stream feeds two paths.

5. Following one order through the system

A customer in Berlin buys headphones for EUR 120 at 10:15:00. Here is everything that happens to that one order.

When	What happens
10:15:00	The simulator, acting as the website, writes the order to the shop's database and publishes it to the Kafka topic *transactions*, encoded in Avro. The Schema Registry checks it matches the agreed format.
10:15:00	ksqlDB evaluates the fraud rules. The order is normal, so no alert fires. Had it been EUR 9,000 placed 20 seconds after login, a HIGH_VALUE_NEW_SESSION alert would be on the *fraud_alerts* topic within a second.
10:15:01	The serving sink writes the order to Postgres. It now shows on Grafana's live orders chart, and the REST API counts it in the customer's recent activity.
10:16:00	Spark's next one-minute batch writes the order, untouched, to the Bronze table for Europe and today's date.
11:00	The hourly Silver job checks it: required fields present, a known customer, a valid currency, a sensible amount and a believable timestamp. It passes. EUR 120 becomes USD at the rate that was current at 10:15, and it's merged into Silver.
Minutes later	Silver's update triggers Gold. dbt builds it into a staging area, tests it, and publishes. The order now counts in Europe's revenue for the day, its customer's lifetime value and its product's ranking.
Next day	Reconciliation confirms the database and Silver agree on how many orders the day had. The order stays in Bronze for 90 days, then moves to cheaper archive storage.

6. The tools: what each does and why it was chosen

Every tool here earns its place by solving a specific problem. For each one: what it is, what it does in StreamCart, why it was chosen, and what else could have done the job.

Generating the data

Python data simulator

What it is. A Python program written for this project that behaves like an online shop and its customers.

What it does here. Produces orders, website visits and stock movements with realistic daily and seasonal patterns, fraud, bots, duplicates and corrupted records. It writes orders to the shop's database and publishes every event to Kafka.

Why it was chosen. Real e-commerce data is private, so a simulator is the only way to have realistic data. Building it in-house means every awkward case the pipeline must handle (late events, duplicates, bad currencies, bots that hide) is deliberately present, and the simulator knows the truth, so the fraud rules can be scored against it.

Moving events: ingestion

Apache Kafka (with ZooKeeper)

What it is. A distributed event log. Producers append events to named topics; any number of consumers read them independently, at their own pace. Events are kept for 7 days. ZooKeeper coordinates the Kafka broker.

What it does here. Holds the three source topics: *transactions*, *clickstream* and *inventory_updates*. Each has 5 partitions, one per market, so markets can be processed in parallel. ksqlDB, Spark and the serving sink all read the same topics without interfering with each other.

Why it was chosen. Kafka decouples the producers from the consumers. If Spark is down for an hour, events simply wait in Kafka and nothing is lost. It lets one stream feed both the speed path and the batch path, which is the core of this design. It's also the industry standard for event streaming.

Alternatives considered. Amazon Kinesis or Google Pub/Sub (managed, but tie the project to one cloud); RabbitMQ (a message queue, which deletes messages once read, so it can't replay or feed several consumers the same way).

Confluent Schema Registry and Avro

What it is. Avro is a compact binary format where every message follows a declared schema. The Schema Registry stores those schemas and gives each version an ID that travels in every message.

What it does here. Holds the contract for each topic. Producers aren't allowed to register new schemas themselves, so an event that doesn't fit the contract is rejected at the source. Changes must be BACKWARD compatible: new fields need a default, so old and new events can always be read together. The orders contract has already evolved once, adding a promo code, without breaking anything.

Why it was chosen. Without a contract, one producer renaming a field silently breaks every consumer downstream. The registry turns that into an error at the moment of change. Avro is also much smaller than JSON.

Alternatives considered. JSON with no schema (simpler, but no protection); Protobuf (an equally good contract format).

Acting in seconds: the speed path

ksqlDB

What it is. A streaming database that runs continuous SQL queries over Kafka topics. A query never finishes: it processes each new event as it arrives and writes results to another topic.

What it does here. Runs the business rules on every order and stock update as it arrives. HIGH_VALUE_NEW_SESSION flags orders over $5,000 placed less than 60 seconds after login (account takeover). VELOCITY_BREACH flags more than 10 orders from one customer in an hour (card testing). GEO_ANOMALY flags an order from a country other than the customer's. A low-stock rule publishes to *restock_required* when stock drops below its reorder point.

Why it was chosen. This is the fix for 'fraud is caught too late'. The rules are written in SQL, which analysts can read and change, rather than in Java or Scala. It runs inside the Kafka ecosystem, so alerts are just more Kafka topics that any system can subscribe to.

Alternatives considered. Apache Flink (more powerful for complex stateful logic, but needs Java or Scala and more operation); Spark Structured Streaming (works, but second-level micro-batches and heavier). The brief allowed ksqlDB or Flink.

Serving sink (Python)

What it is. A small Python service written for this project that consumes Kafka topics and writes them to Postgres.

What it does here. Copies live orders, stock movements, fraud alerts and restock alerts into the *realtime* schema that Grafana and the API read. It commits its Kafka position only after the database write succeeds, and every write is an upsert, so a crash and replay never creates duplicates. It also measures how far behind it is.

Why it was chosen. Dashboards and APIs need a database to query, not a stream. A purpose-built sink is a few hundred lines, easy to test, and keeps exactly the guarantees this project needs.

Alternatives considered. Kafka Connect with a JDBC sink connector (no code, but another service to run and configure).

FastAPI (REST API)

What it is. A modern Python web framework for building APIs, with automatic interactive documentation.

What it does here. Serves live stock levels, low-stock items in order of urgency, and online fraud features for a customer: recent order count, spend, markets used and recent alerts. It connects to Postgres with a read-only login.

Why it was chosen. Dashboards serve people; an API serves other software. A fraud model or the warehouse system can ask for exactly what it needs. FastAPI is fast to build with and documents itself.

Alternatives considered. Flask (older, no built-in validation or docs); a full feature store such as Feast (much heavier).

Storing everything: the lakehouse

MinIO

What it is. An object store that speaks the same API as Amazon S3, run here in a container.

What it does here. Holds the lakehouse: the Bronze and Silver tables as Parquet files, the Spark streaming checkpoints, and an archive bucket for data older than 90 days. Spark and Airflow use a dedicated login that can only reach those two buckets.

Why it was chosen. Object storage is cheap and effectively unlimited, which is why data lakes use it. Using MinIO locally means the same code would run against Amazon S3 in the cloud by changing one address.

Alternatives considered. Amazon S3, Azure Data Lake Storage or Google Cloud Storage (the cloud equivalents); plain local files (not realistic).

Apache Iceberg

What it is. An open table format. It adds database behaviour on top of plain Parquet files in object storage: atomic commits, updates and deletes with MERGE, schema changes, and snapshots you can query as they were in the past.

What it does here. Manages the Bronze and Silver tables. Bronze is partitioned by market and date and stored as Snappy-compressed Parquet. Silver uses MERGE INTO so that reprocessing the same events never creates duplicates. Time travel lets you query a table as it was at an earlier moment. Its catalog, the pointer to each table's current version, lives in Postgres, because S3-style storage can't do the atomic rename a commit needs.

Why it was chosen. Plain Parquet files can't be updated safely, so de-duplication and corrections would be fragile. Iceberg gives warehouse guarantees on cheap storage, which is what makes a *lakehouse*.

Alternatives considered. Delta Lake (similar, strongest with Databricks); Apache Hudi (similar, more complex).

Apache Spark (Structured Streaming and batch)

What it is. A distributed engine for processing large datasets, in batches or as a continuous stream of small batches.

What it does here. Runs three jobs. A streaming job reads Kafka every minute and writes Bronze, decoding each message with the schema version it was written in. The hourly Silver job cleans and validates Bronze into Silver. A daily job extracts the shop's database into Bronze. Nightly maintenance archives old data and compacts small files.

Why it was chosen. Spark is the standard engine for lakehouse work and has first-class Iceberg support. The same code would scale from this single machine to a cluster of hundreds without changes.

Alternatives considered. Pandas or DuckDB (fine at this size, but don't scale out or stream); Flink (stream-first, less suited to the batch cleaning).

Making the data trustworthy

Great Expectations

What it is. A data quality framework. You declare what good data looks like as *expectations*, and it checks every batch against them.

What it does here. Validates every Silver batch before it is written: required fields present, timestamps no more than 5 minutes in the future or 7 days old, currencies valid ISO 4217 codes, amounts positive and below $50,000, and every customer a known user. A failing batch is blocked, not written, and the rejected rows go to a dead-letter table with the reason.

Why it was chosen. Bad data that reaches reports is worse than late data, because people act on it. Great Expectations makes the rules explicit and auditable, and its results are recorded for the observability dashboard.

Alternatives considered. Hand-written checks (no reporting); Soda or dbt tests alone (dbt tests are used too, for Gold).

dbt (data build tool)

What it is. A tool for transforming data with SQL. Each model is a SELECT statement; dbt works out the order to build them, tests them, and generates documentation and lineage.

What it does here. Builds Gold from Silver in layers: staging views, then four dimension tables, three fact tables and six KPI tables. Every model has tests: unique keys, valid values, relationships between tables, sensible ranges. Gold is built and tested in a separate schema and only published if every test passes.

Why it was chosen. dbt brings software engineering to SQL: version control, tests, documentation and reusable code. Analysts can read and change the business logic because it is just SQL.

Alternatives considered. Stored procedures (hard to test or version); Spark SQL jobs (heavier for this size of data).

Serving reports: the warehouse

PostgreSQL

What it is. A reliable open-source relational database.

What it does here. Plays several roles: the shop's transactional database, the Silver copy that dbt reads, the Gold warehouse, the live serving tables, the observability tables, Airflow's own records and the Iceberg catalog. Each service logs in with its own limited role.

Why it was chosen. One familiar, well-supported database keeps the whole platform runnable on a laptop. Gold is small enough that Postgres answers dashboard queries instantly.

Alternatives considered. A cloud warehouse such as Snowflake, BigQuery or Redshift (the production choice at larger scale); Trino over Iceberg (keeps Gold in the lake, but adds a query engine to run).

Running it all on time

Apache Airflow

What it is. A workflow orchestrator. Workflows are Python files (DAGs) that define tasks, their order, schedules and retries.

What it does here. Runs ten DAGs: hourly Silver, exchange rates and external sources; the daily database extract; Gold, which starts whenever Silver updates; nightly Bronze maintenance and housekeeping; a monitor every 15 minutes; and a weekly backfill to replay recent days. Every task retries, and a failure is recorded and alerted.

Why it was chosen. Batch jobs depend on each other and fail sometimes. Airflow makes the dependencies, schedules and retries explicit and visible in one place, with full run history.

Alternatives considered. Cron (no dependencies or retries); Dagster or Prefect (good modern alternatives).

Flyway

What it is. A database migration tool. Schema changes are numbered SQL files applied in order, exactly once per database.

What it does here. Creates and evolves every Postgres schema, table and login through ten versioned migrations, for both new and existing databases. It runs on every start and does nothing if the database is already up to date.

Why it was chosen. Without migrations, a new install and an old one drift apart. Flyway makes every database reach the same state the same way, and records what was changed and when.

Alternatives considered. Liquibase (similar); Alembic (Python-based).

Seeing it

Grafana

What it is. An open-source dashboard tool, strongest for live operational and monitoring dashboards.

What it does here. Shows two dashboards. Real-Time Operations: live orders by market, revenue, the fraud alert feed, a warehouse risk heatmap, pipeline lag and latency, and how accurate the fraud rules are. Pipeline Observability: job success rates, row counts, freshness, quality results, schema changes, failures, reconciliation and volume checks. Both refresh every few seconds and are stored as code.

Why it was chosen. Grafana is built for fast-changing operational data and auto-refresh, which is what fraud and warehouse teams need on a screen.

Alternatives considered. Kibana (log-focused); building into Superset (better for business reporting than live operations).

Apache Superset

What it is. An open-source business intelligence tool for exploring data and building dashboards without code.

What it does here. Serves the business dashboard over Gold: net revenue, order counts, revenue trends by market, average order value, customer retention cohorts, the conversion funnel, conversion by market, top products and revenue by category. It reads Gold through a read-only login.

Why it was chosen. Business teams need self-service exploration: filtering, drilling down and building their own charts. Superset does this and is open source.

Alternatives considered. Power BI, Tableau or Looker (commercial); Metabase (simpler).

Building and shipping it

Docker Compose

What it is. A tool for defining and running multi-container applications from one file.

What it does here. Defines all 24 services, their settings, health checks and start order. One command starts the whole platform. Profiles allow running only the real-time layer, the batch layer or the BI layer.

Why it was chosen. Anyone can run the full platform on one machine with one command, and the configuration is version-controlled.

Alternatives considered. Kubernetes (the production choice, but far heavier for a single machine).

GitHub Actions (CI)

What it is. GitHub's automation service, which runs checks on every push.

What it does here. Lints the Python, validates the Compose file and JSON assets, runs 104 tests, builds Gold with dbt against a real Postgres from empty tables, checks the database logins have only the rights they should, and builds the Docker images.

Why it was chosen. Every change is checked automatically before it can be merged, so a broken model or rule is caught in minutes.

Alternatives considered. GitLab CI or Jenkins (equivalent).

7. The data model

Gold is a **star schema**. *Fact* tables record events and their measures (an order, a session, a day's stock). *Dimension* tables describe the things those events are about (customers, products, dates, markets). *Aggregate* tables hold ready-made KPIs so dashboards stay fast. Dates and hours are in each market's own time zone, so a Tokyo order at 1 a.m. counts on the Tokyo day.

Table	Type	One row per	Business question it answers
dim_customers	Dimension	Customer	Who are our most valuable and most at-risk customers?
dim_products	Dimension	Product	What sells best, and how does it rank in its category?
dim_dates	Dimension	Calendar day	Weekends, seasons and shopping events
dim_regions	Dimension	Market	Currency, time zone and totals per market
fct_orders	Fact	Order	Every non-failed order, linked to all four dimensions
fct_sessions	Fact	Website visit	How long visitors stayed and how far they got
fct_inventory_snapshots	Fact	Product, warehouse and day	Stock on hand, demand and days of cover
agg_daily_revenue	Aggregate	Market and day	How much did we earn, and from what?
agg_fraud_summary	Aggregate	Market and day	How much revenue is at risk from fraud?
agg_inventory_health	Aggregate	Warehouse and day	Which warehouses are at risk of running out?
agg_funnel_conversion	Aggregate	Market and day	Where do visitors drop out before buying?
agg_customer_retention	Aggregate	Weekly cohort and week	Do customers come back?

Table	Type	One row per	Business question it answers
agg_weather_demand	Aggregate	Market and hour	Does rain change demand?

8. Keeping the data trustworthy

A report is only useful if people believe it. StreamCart checks data at every boundary, and keeps evidence of what it rejected.

Check	What it catches	Where
Avro contracts	An event in the wrong shape, at the moment it is produced	Schema Registry
Quality gate	Missing fields, unknown customers, invalid currencies, impossible amounts and timestamps. A failing batch isn't written	Great Expectations, Silver
Dead-letter table	Keeps every rejected row with the reason, for inspection or replay	Silver
Bot filtering	Declared bots by user agent, and hidden bots by click rate (5+ events averaging under 3 seconds apart)	Silver
De-duplication	Duplicate events, within a batch and across reruns, through MERGE on the primary key	Silver
dbt tests	Broken keys, invalid values and missing relationships in every Gold table	Gold
Write, audit, publish	A table that fails its tests is never shown: Gold is built and tested in a staging schema and swapped in only if everything passed	Gold
Reconciliation	Orders in the shop's database that never reached Silver	Daily monitor
Volume checks	A day with under half its usual rows, a sign of silent data loss	Daily monitor
Freshness	A table that has stopped updating	Every 15 minutes
Schema change detection	A changed column or contract version	Every 15 minutes
Fraud rule scoring	How precise and complete each fraud rule is, against the simulator's hidden labels	Every 15 minutes

Each business rule, such as the $5,000 fraud threshold or the list of valid currencies, is defined once in Python. Copies that must live in SQL, YAML or JSON are checked against it by an automated test, so a rule can't be changed in one place and forgotten in another.

9. Running reliably

- **Nothing is lost if a job stops.** Kafka keeps events for 7 days, the Bronze stream resumes from its checkpoint, and Silver tracks a watermark that only moves after a successful write.
- **Reruns are safe.** Every write is idempotent: replaying the same data leaves the tables unchanged. This was proven with two full replays, and the jobs check for duplicate keys after every merge.
- **Late events are kept.** Silver tracks when a row arrived, not when the event happened, so a late event is still picked up, and the 7-day age rule decides whether it's acceptable.
- **Gold runs when Silver is ready.** It is triggered by Silver's update, not a fixed time that might come first.
- **Schemas can evolve mid-stream.** Each event is decoded with the version it was written in; new fields become new columns automatically.
- **Data doesn't grow forever.** Bronze moves to an archive after 90 days, small files are compacted, old snapshots expire, and monitoring and Airflow history are pruned.

- **People hear about failures.** Failed tasks and volume drops are recorded for Grafana and posted to a Slack or Mattermost webhook.

10. Security

The principle is **least privilege**: every service gets only the access its job needs, so a bug or a leaked password in one place can do limited damage.

Component	How access is limited
Postgres	Pipeline services use an application login that can't create roles or databases. Dashboards and the API use a read-only login. Superset's own metadata has its own login. The superuser is used only by Postgres itself and the migrations
Kafka	Every client logs in with its own user, and access control lists allow only what it uses: the simulator can write the source topics but not read them, Spark and the sink can read but not write, and a client with no login gets nothing
MinIO	Spark and Airflow's login can read and write objects in the two lakehouse buckets only. It can't create or delete buckets or manage users, and those containers never receive the root password
Secrets	Passwords live in a .env file that is never committed to git

11. What it looks like

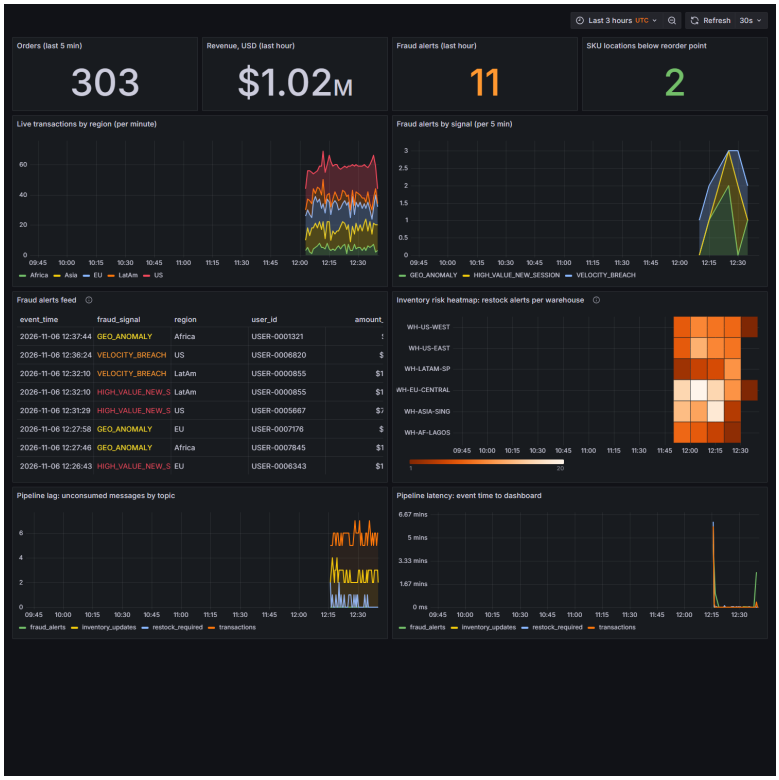


Figure 2. Grafana, Real-Time Operations: live orders, the fraud alert feed, warehouse risk and pipeline lag.

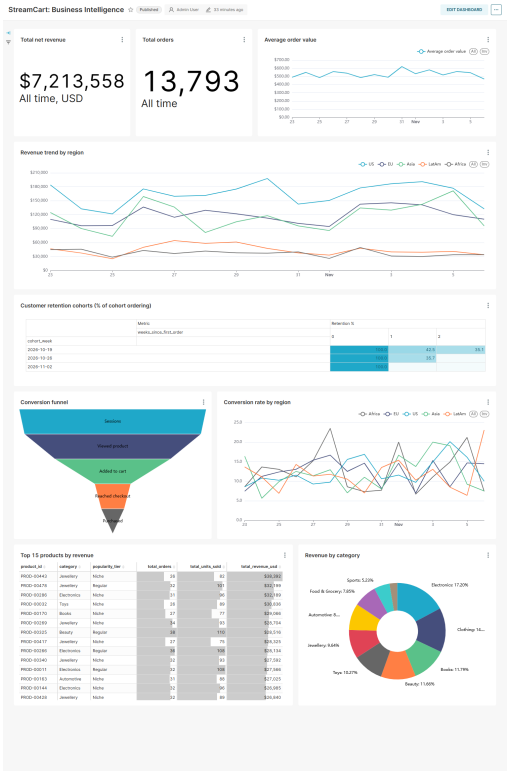


Figure 3. Superset, Business Intelligence: revenue trends, retention cohorts, the conversion funnel and top products.

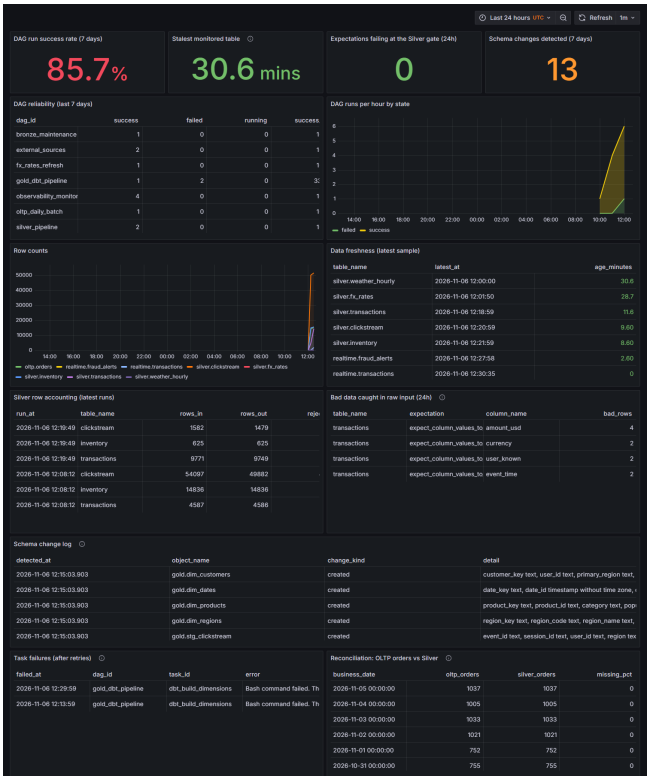


Figure 4. Grafana, Pipeline Observability: job reliability, row counts, freshness and quality results.

12. Results from testing

Everything below was run against the live platform, and the final version passed a complete install from empty storage.

Check	Result
Clean install	All 24 services start with one command, and every setup job succeeds
Automated tests	104 of 104 pass, and CI runs them on every push with a dbt build and image builds
Order volume	63 to 64 orders a minute at a weekend peak, matching the design (over 50,000 a day)
Silver to Gold	13,126 valid orders in Silver and 13,126 in Gold on a fresh install
Bad data	Of 53,820 website events in the seeded history, 49,504 were kept after bots and bad rows were removed
Idempotency	After two full replays, row counts still equal distinct IDs everywhere
Write, audit, publish	A deliberately failing test stopped the build, and nothing reached the live tables
Alerts	A real failure reached a test webhook with the task, the error and a link to its log
Schema evolution	A new contract version was registered mid-stream with no lost or duplicated events
Time zones	About 30% of Asia's orders moved to their correct local day
Security	Each login was refused when it tried to do something outside its role
Recovery	The platform resumed by itself after the machine slept and after a Kafka restart

13. Limits, and what production would change

Limit today	Why	What production would do
One of each server	One Kafka broker, one Spark worker and one Postgres, to fit on a laptop	Replicated Kafka, a Spark cluster and a managed, replicated database
Gold in Postgres	Simple and fast at this size	A cloud warehouse, or a query engine such as Trino over Iceberg
No encryption in transit	Everything runs on one machine's private network	TLS on Kafka, Postgres and object storage
Simulated data	There's no real shop	Real sources. The fraud scores and weather effect show the pipeline can measure them, not how real fraud or demand behave
Fixed fraud rules	Rules are explainable and fast	A trained model, using the API's features and compared against the rules' scores
Inventory API isn't independent	It serves stock the simulator already streamed	Pull from the real warehouse system
Alerts go to a chat webhook	No on-call rota exists	A paging service with routing by severity

14. Glossary

Term	Meaning
ACL	Access control list: rules saying which user may do what to which resource
Avro	A compact binary data format where every record follows a declared schema
Batch path	The slower, thorough path that cleans, validates and models data for reporting
Bronze, Silver, Gold	The three medallion layers: raw, cleaned, and business-ready
DAG	Directed acyclic graph: an Airflow workflow of tasks and their order
Dead-letter table	Where rejected rows are kept with the reason they failed
Idempotent	Running something twice has the same result as running it once
Lakehouse	Cheap object storage with database guarantees added by a table format like Iceberg
Partition	A slice of a topic or table, such as one market or one day, processed independently
Precision and recall	Of the alerts raised, the share that were real fraud; and of the real fraud, the share that raised an alert
Speed path	The fast path that turns events into alerts and live dashboards within seconds
Star schema	Fact tables of events surrounded by dimension tables that describe them
Time travel	Querying a table as it was at an earlier point in time
Watermark	A marker of how far a job has processed, so the next run starts where it stopped
Write, audit, publish	Build data somewhere hidden, test it, and only then make it visible